Multi-Database Synchronization System with RAM-Based Operation History
Project Analysis and Detailed Design

2026-09-06

Contents

1 Executive summary

This document fully describes the Multi-Database Synchronization System implemented in the repository aadityajoshi205/NOSQL_Project_extension. The system provides a RAM-staged temporary memory layer per database to collect, track, and undo changes, and a sync engine to write latest states to heterogeneous databases (MongoDB, Hive, PostgreSQL). It supports SET, GET, DELETE, UNDO, MERGE, FULL_SYNC, COMMIT, ROLLBACK, writes per-database operation logs (oplogs.*), and supports multithreaded synchronization for efficiency.

2 System design (detailed)

This section explains the system architecture, the RAM-inspired temporary memory, caching, and how the system manages synchronization across heterogeneous databases.

2.1 Design goals and inspiration

- Track operations in-memory (RAM) per database to allow fast staging of writes, rollbacks, undo, and local merges without immediately forcing disk/database writes.
- Enable synchronization across heterogeneous DBs using per-key timestamps and oplogs.
- Use multi-threading for parallel writes when committing/staging to multiple databases, minimizing total sync time.

The RAM layer is explicitly inspired by system RAM: it is transient, fast, and holds recent modifications until a COMMIT or ROLLBACK is performed.

2.2 High level components

- User input / test-case parser: parses an oplogs-style input file with lines like 'MONGODB.SET((sid,cid), grade)' and controls the high-level operations (parse_testcase.py and parse_testcase_multithreading.py).
- RAM layer: in-memory dictionary per database storing a per-key history item (timestamp, value, delete flag). See README's data structure and implementation in parsing code.
- Merge engine: logic to merge two RAMs (merge.py) using timestamps to pick latest values per key.
- Sync engine: writes latest value per key from RAM to actual DB connectors (sync.py).
- DB connectors: classes that encapsulate operations for MongoDB (MongoDB_connect.py), PostgreSQL (postgresql_connector.py), and Hive (hive.py – connector present in repo).
- Oplogs: per-database log files (oplogs.mongodb, oplogs.postgresql, oplogs.hive) that record SET/DELETE/GET operations (text files written by parser/merge logic).

2.3 RAM representation

Each database has a RAM cache (dictionary) with keys being the composite primary key (student_id, course_id) and values being a tuple/list representing the latest state and history flag(s). README and code use:

```
Dict[Tuple[student_id, course_id], List[Tuple[grade, timestamp, delete_flag]]]
```

In the concrete code shipped in this repository, the RAM entries are represented as:

- parse_testcase.py: globals()[DB + "_cache"][(student_id, course_id)] = [timestamp, grade, delete_flag]
- parse_testcase_multithreading.py uses a simpler [timestamp, grade] entry for some flows (no explicit delete flag in that path), but main implementation (parse_testcase.py) uses the triple with delete flag.

Semantics:

- timestamp: precise timestamp (microsecond/nanosecond precision string) used for conflict resolution in merges.
- grade: the stored value for that key.
- delete_flag: 1 for deleted, 0 for active.

2.4 Operation semantics (RAM)

- SET: pushes a new tuple ([timestamp, value, delete_flag=0]) for the key (or creates new key). The parser writes to per-db oplog and updates the RAM.
- GET: fetches from RAM top entry if present and not deleted, otherwise queries the DB connector. GET is primarily a read that also logs the access in per-db oplog.

- DELETE: marks the key as deleted by storing [timestamp, last_value, delete_flag=1] in RAM rather than physically removing it immediately.
- UNDO: uses a separate `undo` stack collected during parsing to revert changes by restoring previous RAM entries. This supports per-key undo as recorded in the undo structure.
- MERGE: unidirectional merge from source RAM to target RAM. For each key in source, if absent in target it is copied; if present, timestamps compared and target replaced if source is newer. The merge also logs SET/DELETE changes to target's oplog.
- FULL_SYNC: merges all RAMs together by combining pairwise merges and propagating latest values across all caches; it is performed in RAM only and is non-reversible.
- COMMIT: writes the latest state for each key from a RAM into the corresponding physical database (via sync.py). After commit, the RAM cache is cleared for that DB.
- ROLLBACK: clears the RAM cache for each DB without writing to disk (discard staged changes).

2.5 Conflict resolution

Timestamp-based: timestamp strings are created with nanosecond precision using time.time_ns() and formatted into an ISO-like string (see parse_testcase.get_precise_timestamp). During MERGE, keys are compared by timestamp and the newest timestamp wins. The implementation compares the stored timestamp strings lexicographically (they are ISO-ish and include zero-padded nanoseconds so lexical comparison works).

2.6 Threading and concurrency

- The sync to databases (COMMIT) can be parallelized by launching one thread per database: parse_testcase_multithreading.py uses concurrent.futures.ThreadPoolExecutor and submits sync(db_handler, snapshot) tasks. Each sync task iterates keys and calls db_handler.set or db_handler.delete.
- To avoid DB-level write contention, the system defers DB writes until sync/commit rather than writing during MERGE operations; this avoids multiple threads trying to lock/update the same DB during many MERGEs and reduces race conditions.

3 Important code parts (with explanations)

Below are the repository's most relevant source files (excerpts included) and a short explanation of each.

3.1 main.py

Purpose: small bootstrap that constructs DB handlers and kicks off parsing of the test-case file.

Listing 1: main.py

```
import csv
from collections import defaultdict
import json
```

```
from MongoDB_connect import MONGODBHANDLER
from postgresql_connector import POSTGRESQLHANDLER
from hive import HIVEHANDLER
from parse_testcase import parse_testcase_file
from parse_testcase_multithreading import
   parse_testcase_file_multithreading
from hive import HIVEHANDLER

Databases=["MONGODB", "POSTGRESQL"]

db_handlers=[]

for db in Databases:
    globals()[db.lower()+"_handler"]=globals()[db+"HANDLER"]()
    db_handlers.append(globals()[db.lower()+"_handler"])
file_path = input("Enter the path of the testcase file: ").strip()
parse_testcase_file(
    file_path=file_path,
    db_handlers=db_handlers,
    Databases=Databases
)
```

Notes:

- The script dynamically constructs handler instances by expecting classes named MONGODBHANDLER, POSTGRESQLHANDLER, HIVEHANDLER etc., and registering them into globals using a naming convention.
- The list Databases controls which databases are involved in the run.

3.2 MongoDB connector (MongoDB_connect.py)

This class wraps pymongo operations: connect, set (update/upsert), get, delete, and bulk insert.

Listing 2: MongoDB_connect.py

```
from pymongo.mongo_client import MongoClient
from pymongo.server_api import ServerApi
import csv

class MONGODBHANDLER:
    def __init__(self, uri: str = "...", primary_keys=None):
        self.uri = uri
        self.client = None
        self.collection = None
        self.db = None
        self.primary_keys = primary_keys
        self.connect()

    def connect(self):
        try:
            self.client = MongoClient(self.uri, server_api=ServerApi('1'))
            self.client.admin.command('ping')
            self.db = self.client["university_db"]
            self.collection=self.db["student_course_grades"]
```

```
            print("Pinged your deployment. You successfully connected to MongoDB!")
        except Exception as e:
            print("Connection failed:", e)

    def set(self, db_name: str, collection_name: str, pk: tuple, value: str, ts: int):
        student_id, course_id = pk
        query = {"student-ID": student_id, "course-id": course_id}
        update = {"$set": {"grade": value}}
        result = self.collection.update_one(query, update, upsert=True)
        if result.matched_count == 0:
            print("No matching document found to update. Inserted new record")
        elif result.modified_count == 1:
            print("Document updated successfully.")
        else:
            print("Document matched but no change was made.")

    def get(self, db_name: str, collection_name: str, pk: tuple):
        db = self.client[db_name]
        collection = db[collection_name]
        query = {"student-ID": pk[0], "course-id": pk[1]}
        return collection.find_one(query)

    def delete(self, db_name: str, collection_name: str, pk: tuple):
        db = self.client[db_name]
        collection = db[collection_name]
        query = {"student-ID": pk[0], "course-id": pk[1]}
        result = collection.delete_one(query)
        return result.deleted_count > 0

    def bulk_insert_students_from_csv(self, db_name: str, collection_name: str, csv_path: str):
        student_records = []
        with open(csv_path, newline='') as csvfile:
            reader = csv.DictReader(csvfile)
            for row in reader:
                student_records.append(row)
        if student_records:
            db = self.client[db_name]
            collection = db[collection_name]
            result = collection.insert_many(student_records)
            print(f"Successfully inserted {len(result.inserted_ids)} student records.")
```

Notes:

- Uses update_one with upsert for SET semantics (create or replace grade).
- DELETE uses delete_one on the composite key.
- There is a placeholder URI in the repo file (real URI should be configured securely when deploying).

3.3 PostgreSQL connector (postgresql_connector.py)

Provides connect, set (update-or-insert), get, delete, and disconnect. It uses psycopg2 and parameterized queries:

Listing 3: postgresql_connector.py

```
import psycopg2
from psycopg2 import sql

class POSTGRESQLHANDLER:
    def __init__(self, host: str = "localhost", port: int = 5432, database
        : str = "doshte", user: str = "nande", password: str = "050309",
        primary_keys=None):
        self.host = host
        self.port = port
        self.database = database
        self.user = user
        self.password = password
        self.primary_keys = primary_keys
        self.connection = None
        self.cursor = None
        self.connect()

    def connect(self):
        try:
            self.connection = psycopg2.connect(
                host=self.host,
                port=self.port,
                dbname=self.database,
                user=self.user,
                password=self.password
            )
            self.cursor = self.connection.cursor()
            print("␣Connected␣to␣PostgreSQL␣successfully!")
        except Exception as e:
            print("␣Connection␣failed:", e)

    def set(self, database:str, table: str, pk: tuple, value: str, ts: int
        ):
        student_id, course_id = pk
        update_query = f"""UPDATE␣{table}␣SET␣grade␣=␣%s␣WHERE␣"student-ID
            "␣=␣%s␣AND␣"course-id"␣=␣%s;"""
        self.cursor.execute(update_query, (value, student_id, course_id))
        if self.cursor.rowcount == 0:
            print("No␣matching␣row␣found␣to␣update.␣Inserting␣new␣row.")
            insert_query = f"""INSERT␣INTO␣{table}␣("student-ID",␣"course-
                id",␣grade)␣VALUES␣(%s,␣%s,␣%s);"""
            self.cursor.execute(insert_query, (student_id, course_id,
                value))
        elif self.cursor.rowcount == 1:
            print("Row␣updated␣successfully.")
        else:
            print("Multiple␣rows␣updated␣(unexpected).")
        self.connection.commit()
```

```
43
44      def get(self,database_name: str, table_name: str, pk: tuple):
45          try:
46              pk_columns = ("student-ID", "course-id")
47              if len(pk) != len(pk_columns):
48                  raise ValueError("PK mismatch")
49              where_clause = sql.SQL(" AND ").join(
50                  sql.Composed([sql.Identifier(col), sql.SQL(" = %s")]) for
                        col in pk_columns
51              )
52              query = sql.SQL("SELECT * FROM {table} WHERE {conditions}").
                    format(
53                  table=sql.Identifier(table_name),
54                  conditions=where_clause
55              )
56              self.cursor.execute(query, pk)
57              result = self.cursor.fetchone()
58              if result:
59                  print(f"Found data: {result}")
60                  return result
61              else:
62                  print("No data found for the given PK.")
63                  return None
64          except Exception as e:
65              print("Get operation failed:", e)
66              return None
67
68      def delete(self, database: str, table: str, pk: tuple):
69          student_id, course_id = pk
70          delete_query = f"""DELETE FROM {table} WHERE "student-ID" = %s AND
                 "course-id" = %s;"""
71          self.cursor.execute(delete_query, (student_id, course_id))
72          if self.cursor.rowcount == 0:
73              print("No row found to delete.")
74          elif self.cursor.rowcount == 1:
75              print("Row deleted successfully.")
76          else:
77              print("Multiple rows deleted (unexpected).")
78          self.connection.commit()
79
80      def disconnect(self):
81          try:
82              if self.cursor:
83                  self.cursor.close()
84              if self.connection:
85                  self.connection.close()
86              print(" Disconnected from PostgreSQL.")
87          except Exception as e:
88              print("Disconnection failed:", e)
```

Notes:

- `set` attempts an UPDATE first and then INSERT if no row updated; a common upsert pattern when there is no ON CONFLICT configured.

- Parameterized queries used via psycopg2 to avoid SQL injection.

3.4 Sync engine (sync.py)

Realizes commit semantics by iterating current RAM cache and applying the latest tuple to the DB handler.

Listing 4: sync.py

```
def sync(db_handler,db_cache):
    for key, value in db_cache.items():
        pk= key
        timestamp, grade, delete_flag = value
        if delete_flag:
            db_handler.delete("university_db", "student_course_grades", pk
                )
        else:
            db_handler.set("university_db", "student_course_grades", pk,
                grade, timestamp)
```

Notes:

- Sync writes only the latest tuple for each key to the physical DB. This keeps DB writes minimal and predictable.
- When used concurrently (one thread per DB), the commit time is reduced for multi-DB deployments.

3.5 Merge engine (merge.py)

Merges source RAM into destination RAM using timestamp comparisons and logs the applied operations to the target oplog.

Listing 5: merge.py

```
def merge(db_cache_1,db_cache_2,db1):
    temp=[]
    for key in db_cache_2.keys():
        if key not in db_cache_1.keys():
            db_cache_1[key] = db_cache_2[key]
            logger=open('oplogs.' + db1.lower(), 'a')
            latest_ts, latest_value, delete_flag = db_cache_2[key]
            pk = key
            if delete_flag:
                logger.write(f"{latest_ts},␣{db1}.DELETE(({pk[0]},{pk[1]})
                    )\n")
            else:
                logger.write(f"{latest_ts},␣{db1}.SET(({pk[0]},{pk[1]}),␣{
                    latest_value})\n")
        else:
            if db_cache_2[key][0] > db_cache_1[key][0]:
                print("raplacing␣values")
                temp.append((db1,db_cache_1[key][0],key[0],key[1],
                    db_cache_1[key][1],db_cache_1[key][2]))
                db_cache_1[key] = db_cache_2[key]
```

```
                logger=open('oplogs.' + db1.lower(), 'a')
                latest_ts, latest_value, delete_flag = db_cache_2[key]
                pk = key
                if delete_flag:
                    logger.write(f"{latest_ts}, {db1}.DELETE(({pk[0]},{pk
                        [1]}))\n")
                else:
                    logger.write(f"{latest_ts}, {db1}.SET(({pk[0]},{pk
                        [1]}), {latest_value})\n")
    return (db_cache_1,temp)
```

Notes:

- The function returns the updated destination cache and a `temp` list used for undo information.
- Timestamps determine which value to keep.

3.6 Parser: parse_testcase.py

Implements the line-by-line test case parser that recognizes TIMESTAMP-prefixed lines, and commands such as SET/GET/DELETE/MERGE/COMMIT/ROLLBACK/UNDO/FULL_SYNC. It updates RAM caches and writes per-db oplog files.

Key behaviors:

- Initializes per-db caches as globals named e.g. "MONGODB_cache".
- Writes ops to 'oplogs.mongodb', 'oplogs.postgresql', etc.
- COMMIT triggers sync(db_handler, db_cache) for each DB and then clears the cache.

(Refer to the code in the repository for the complete parser. It includes get_precise_timestamp() that formats nanosecond-resolution timestamps.)

3.7 Parser with multithreading: parse_testcase_multithreading.py

Very similar to parse_testcase.py, but uses a ThreadPoolExecutor to submit sync tasks in parallel at the end of parsing and when performing COMMIT-like syncs. It queues sync futures and waits on them at shutdown.

Notes:

- Uses `executor.submit(sync, db_handlers[i], globals()[db + "_cache"])` to perform DB writes concurrently.
- This design creates one thread per DB during commit and allows parallel DB writes.

4 Entire user workflow

This repository is driven by a text-based test-case input file ("oplog" style). The typical user flow is:

1. Prepare an operation sequence file (the test-case) with lines of the forms:
    - TIMESTAMP, DB.SET((student,course), grade)

- DB.GET(student,course)
- DB.DELETE(student,course)
- DB.MERGE(OtherDB)
- FULL_SYNC
- COMMIT
- ROLLBACK
- UNDO

2. Run the main driver (main.py). It will:
   - Instantiate DB handlers for names listed in Databases (in main.py).
   - Ask for the path to the test-case file via input prompt.
   - Invoke parse_testcase_file which reads the test-case and processes each line.
3. As parse_testcase processes operations it:
   - Updates per-DB RAM caches.
   - Writes a per-database operation log file (oplogs.) that mirrors user actions and the derived RAM updates.
   - For COMMIT: calls sync to push latest states to the physical DBs, then clears RAM caches.
   - For ROLLBACK: clears RAM caches (no DB writes).
   - For MERGE/FULL_SYNC: manipulates RAM caches only (no DB writes) and logs changes to oplogs.
4. For multithreaded runs: parse_testcase_multithreading.py will submit sync operations to a thread pool to write each DB concurrently for commits.
5. After the run, user can inspect per-DB oplogs and the databases to verify committed changes.

5 Tech stack

- Language: Python (repository is Python-only). The repo's language composition shows Python at 100%.
- Databases demonstrated:
  - MongoDB (MongoDB_connect.py)
  - Hive (hive.py – present in repository)
  - PostgreSQL (postgresql_connector.py)
- Libraries & tools:
  - pymongo for MongoDB
  - psycopg2 for PostgreSQL
  - concurrent.futures for multithreading
  - Standard Python modules: csv, datetime, time, re, json, collections

6 All features of the project

This section enumerates the explicit features implemented in the code and README:

- RAM-based temporary memory per database with per-key history.
- SET/GET/DELETE operations recorded into per-DB oplogs.
- UNDO support via an undo stack collected during parsing.
- MERGE (unidirectional) merges RAM caches using timestamps; logs applied changes to oplogs.
- FULL_SYNC: multi-RAM merge across all DBs (RAM-only).
- COMMIT: flush RAM → persistent DB via sync (multi-threaded support).
- ROLLBACK: clear RAM caches (discard staged changes).
- Multithreading: ThreadPoolExecutor used to sync multiple DBs concurrently (one thread per DB).
- Bulk CSV insertion support (MongoDB connector provides a bulk insert method).
- Extensibility: add new DB connectors and register DB names in main.py "Databases" list; the main driver will instantiate handlers dynamically by naming convention.

7 Data structures and algorithms used

7.1 Data structures

- Python dict (hash map) for RAM caches: key = (student_ID, course_ID) tuple, value = list/tuple representing latest state and flags. Fast O(1) amortized lookups, inserts, and updates by key.
- Per-key stacks/histories (conceptually): README says each key holds a stack/list of tuples (grade, timestamp, delete_flag) to hold historical versions; the code in parse_testcase uses an undo list and stores the latest tuple inside the cache. The `undo` mechanism collects prior tuples for restoration.
- Per-database oplog files (text files) for append-only logs of operations: easy for offline auditing and reconstruction.

7.2 Algorithms / complexity

- SET/DELETE/GET in RAM: O(1) per operation (dict insert/update/read).
- MERGE: iterates keys in source RAM; for each key checks membership in destination (O(1)), compares timestamps (string compare) and possibly replaces destination entry. Complexity O(N) where N is number of keys in source RAM. Writes to oplog are O(1) per key (append to file).
- FULL_SYNC: repeated MERGEs across all DB caches => effectively O(total keys across all DB caches) with some repetition depending on merge order.

- COMMIT (sync): iterates over keys in RAM and performs DB I/O per key. Complexity dominated by DB I/O; using threading reduces end-to-end time across DBs but not per-DB total I/O.
- UNDO: uses an undo stack/list maintained by the parser. Popping and restoring entries is O(1) per undo restore.
- Multithreading: simple parallelism by executing independent sync tasks concurrently (no distributed transactions). The approach assumes each DB's writes are independent for the purposes of performance.

8 Extensibility to other DBs

To add another database:

1. Implement a handler class named DBNAMEHANDLER (e.g., COUCHDBHANDLER) exposing:
   - connect()
   - set(database, table/collection, pk, value, timestamp)
   - get(database, table/collection, pk)
   - delete(database, table/collection, pk)
   - optional bulk insert/disconnect as required
2. Add the DB name to the Databases list in main.py, for example Databases = ["MONGODB", "POSTGRESQL", "NEWDB"].
3. Ensure the handler class is imported into main.py scope or placed in a module that main.py imports so the dynamic globals lookup works:
   - main.py uses `globals()[db + "HANDLER"]()` to instantiate each handler.
4. If the DB uses different primitives for upsert or atomic operations, implement set() accordingly. The sync engine expects that set() will bring the DB row/document to the supplied value (upsert semantics).

Design benefits:

- Connector abstraction decouples the sync logic from database specifics.
- RAM-based staging means new connectors only need to support the simple CRUD primitives.

Caveats:

- No cross-DB distributed transaction or two-phase commit; COMMIT is per-run and multi-threaded but not atomic across all DBs. If atomic cross-DB commit is required, integration with a distributed transaction coordinator or compensating transactions would be needed.

9 All operations supported and how they are realized

Below is the complete list and the data structures/algorithms that implement them:

9.1 SET(DB, PK, value, ts):

- Implementation: parse_testcase.py updates globals()[DB + "_cache"][PK] = [timestamp, grade, 0]; appends to oplogs.DB; records old state onto undo stack as needed.
- Data structures: dict insertion/update.
- Complexity: O(1) in RAM; DB I/O only at commit time.

9.2 GET(DB, PK):

- Implementation: check RAM cache; if present and not deleted return that value; otherwise call DB handler.get(). Writes to oplogs.
- Data structures: dict lookup; DB read if not cached.
- Complexity: O(1) RAM hit; DB-dependent otherwise.

9.3 DELETE(DB, PK):

- Implementation: push [timestamp, last_value, 1] into RAM cache for key; append to oplogs. At commit time, sync() sees delete_flag and calls db_handler.delete().
- Data structures: dict update; undo stack stores prior version.
- Complexity: O(1) in RAM; DB delete at commit.

9.4 UNDO():

- Implementation: parser maintains an undo list that stores prior entries (per operation) and pops the saved tuple(s) to restore prior RAM state.
- Data structures: list used as a stack.
- Complexity: O(k) where k is number of restored items (usually small per undo).

9.5 MERGE(A.MERGE(B)):

- Implementation: merge.py iterates keys in B's cache and:
  - if key missing in A: copy entry and append a SET/DELETE line in A's oplog (no DB writes).
  - if key exists in A: compare timestamps, if B's timestamp newer then replace A's entry and append to A's oplog; save prior value to a temp list for undo.
- Data structures: dict membership checks and assignment.
- Complexity: O(|B|) time.

9.6 FULL_SYNC():

- Implementation: pairwise merges between caches to ensure latest data per key is present in each RAM. Non-reversible. Uses merge operations and is RAM-only (no DB writes).
- Complexity: O(total keys across caches).

9.7 COMMIT():

- Implementation: For each DB cache, call sync(db_handler, db_cache).
  - sync() iterates keys and for each key calls db_handler.set() or db_handler.delete() depending on delete_flag.
  - parse_testcase_multithreading.py can submit each DB's sync to the thread pool so multiple DB writes happen in parallel.
- Data structures: iteration over dict items, DB connector calls (I/O dominated).
- Complexity: O(number of keys) plus DB I/O.

9.8 ROLLBACK():

- Implementation: clear each DB cache (globals()[DB + "_cache"].clear()), discard staged changes. No DB operations performed.
- Complexity: O(1) to clear references; underlying memory freed by Python GC as appropriate.

10 Operation logs: universal oplogs file and per-database splitting

This section documents a behavior of the operation-log mechanism that is described in the README but not previously called out explicitly in this report.

- To simulate user activity that would otherwise come from a frontend, the system is driven by a single *universal* oplogs input file, following the operation syntax described in Section 5 (SET/GET/DELETE/MERGE/FULL_SYNC/COMMIT/ROLLBACK/UNDO, each optionally prefixed with a timestamp).
- As this universal file is parsed, each operation is inspected to determine which database it targets, and is then appended into that database's own oplog file (oplogs.mongodb, oplogs.postgresql, oplogs.hive) – i.e., the single input stream is split into several per-database logs, one per participating DB, by filtering out only the operations relevant to that DB.
- Consequently, each per-database oplog is a self-contained record of every SET, DELETE, and UNDO applied to that database, tagged with a timestamp and the affected value, which allows a user to review that database's change history independently of the others after a run completes.

11 Challenges faced and solutions used to solve them

This section reproduces, in expanded form, the concurrency issue the project team encountered during development and how it was resolved – content present in the README's "Challenges faced" notes but not previously included in this report.

- **Initial design.** In an early version of the system, a RAM-to-database synchronization thread was launched immediately after every single `A.MERGE(B)` call, rather than being deferred to a later point in execution.

- **Problem observed.** Whenever two or more consecutive MERGE calls shared the same recipient database – for example, back-to-back calls such as `A.MERGE(B)` followed immediately by `A.MERGE(C)` – the program would stop making progress and hang indefinitely instead of completing.
- **Root cause.** In that scenario, more than one sync thread ended up trying to acquire a write lock on the *same* destination database at (nearly) the same time. Each thread held a lock the other needed, so both threads waited on each other forever: a classic deadlock caused by uncoordinated concurrent locking of a shared resource.
- **Fix adopted.** The team moved database synchronization out of the per-MERGE code path entirely. Instead, synchronization is now triggered only once, at the very end of processing a test case (or after an explicit SYNC/COMMIT step), and is parallelized using exactly *one* thread per destination database rather than one thread per MERGE call. Because at most one thread ever holds a lock on a given database at a time, the deadlock is eliminated, while the system still keeps the performance benefit of writing to multiple independent databases in parallel.

12 Limitations and important notes

- Full-sync and rollback are destructive (FULL_SYNC irreversible and ROLLBACK discards staged changes). The README calls these out.
- No cross-DB transaction atomicity: COMMIT is not a distributed two-phase commit. If one DB commit fails while others succeed, there is no automatic global rollback; user must rely on oplogs and manual reconciliation.
- Timestamp precision: in practice, generating very high precision timestamps caused some test issues; the README mentions test flakiness when using microsecond/more precise timestamps for merge semantics in testing. The system uses nanosecond-precision formatted strings to help deterministic ordering but tests must account for timestamp precision.
- Configuration / secrets: MongoDB connection URI and PostgreSQL credentials are present in the code and should be managed via environment variables or a secure secrets mechanism when deploying. Avoid committing secrets into source control in production.
- Real-time consistency between databases is only guaranteed after a COMMIT; before that point, different databases' RAM caches may legitimately disagree with each other and with disk state, since MERGE/FULL_SYNC only affect RAM.

12.1 Technical highlights and the microsecond-precision investigation

This subsection expands on the README's "Technical Highlights" notes regarding timestamp precision, which give useful context for the Conflict Resolution discussion in Section 2.5.

- The overall design is multithreaded, and the RAM layer is credited with three concrete benefits: efficient change tracking, quick rollbacks, and seamless merge/sync; the undo-stack model is what enables per-key operation history to be reconstructed.

- During testing at microsecond timestamp precision, some MERGE test cases intermittently failed. Investigation showed that under the system's low per-operation processing latency, it was possible for more than one operation to be processed within the same microsecond, so two logically distinct operations could receive an identical timestamp. This made merge ordering ambiguous for those colliding keys, since the merge logic relies on strict timestamp comparison to decide which value is "newer."
- Regenerating timestamps at nanosecond precision resolved the flakiness, because collisions at that finer resolution did not occur for the test workloads used. The team notes that the microsecond-level collisions were themselves incidental evidence that the system was processing operations with very low latency.

13 Additional repository notes and recommended next steps

- Carefully externalize secrets (DB URIs, credentials) to environment variables or configuration files not committed to source control.
- Add unit/integration tests around merge/commit semantics to validate behavior under concurrent merges and partial commit failures.
- Consider adding a safe two-phase commit or compensating transaction approach for users who need atomic cross-DB commits.
- Add clearer documentation for the expected format of input test-case files (examples exist in the repo under example_testcase.in) and include schema definitions for the required DB tables/collections.

14 Appendix: selected file excerpts

For convenience, this section reproduces the key source files (already shown earlier in the document). If you prefer, I can instead produce a single PDF with syntax-highlighted code excerpts and diagrams or produce additional documentation pages per module.